

МОДУЛЬ 1

АДМИНИСТРИРОВАНИЕ LINUX

ЗАГРУЗКА LINUX

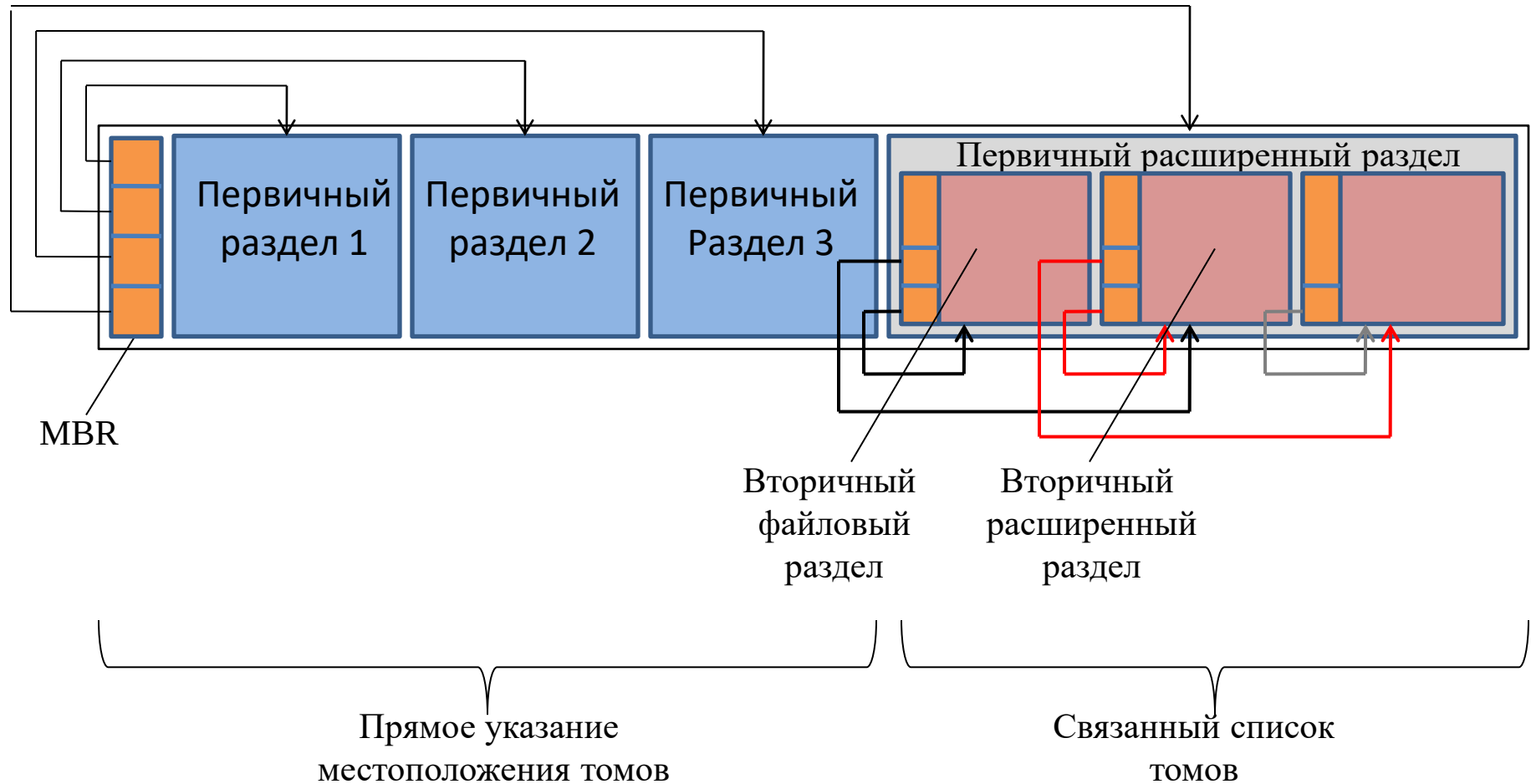
ЭТАПЫ ЗАГРУЗКИ ОС



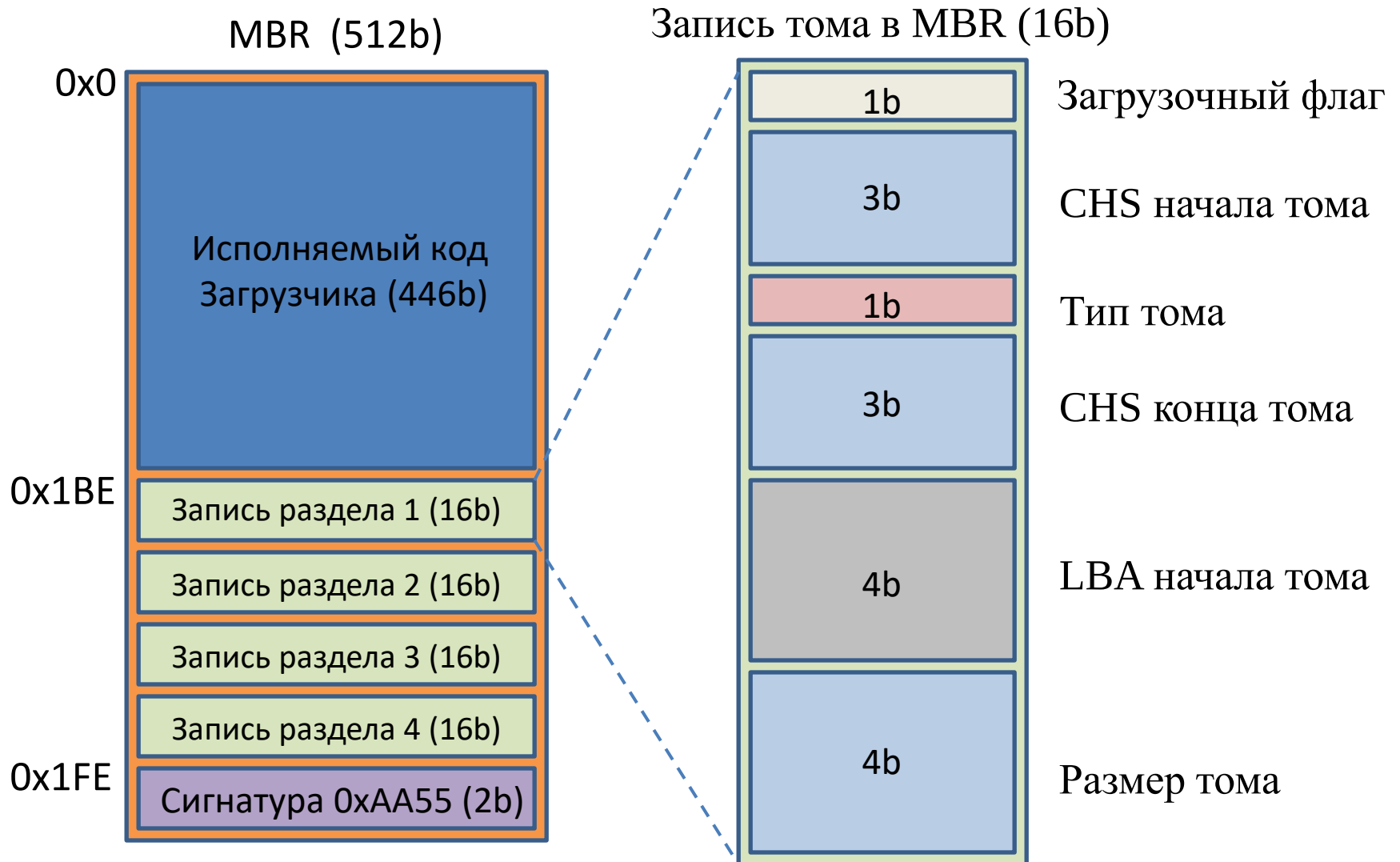
ЭТАПЫ ЗАГРУЗКИ LINUX

- Загрузка загрузчика (*lilo*, *grub*, пр.)
- Загрузка RAM-диска
- Загрузка ядра *vmlinuz-X.XX.X-X*
- Запуск корневого процесса
- Переход в уровень доступа (*runlevel*)
- Запуск startup-скриптов

ДИСКОВАЯ АДРЕСАЦИЯ



СТРУКТУРА MBR



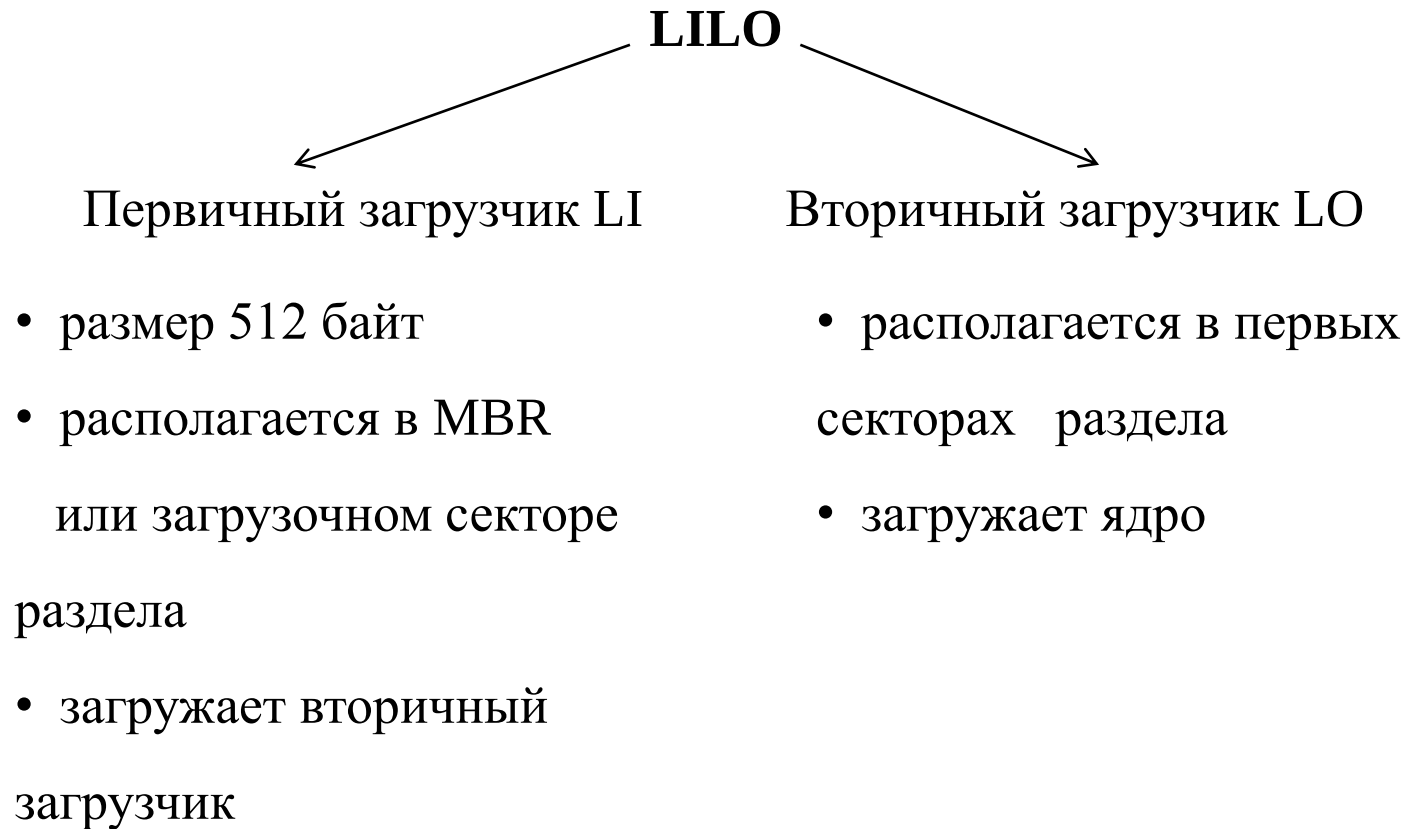
ДИРЕКТОРИЯ /BOOT

Директория содержит файлы и конфигурации, используемые при загрузке Linux

Содержимое директории:

- `vmlinuz-X.XX.X-X-default` – сrio-архив ядра
- `vmlinux-X.XX.X-X-default.gz` – gzip-архив ядра
- `vmlinuz` – символическая ссылка на `vmlinuz-X.XX.X-X-default`
- `initrd-X.XX.X-X-default` – RAM-диск для основного ядра
- `System.map-X.XX.X-X-default` – символическая таблица ядра
- `backup_mbr` – резервная копия MBR
- `config-X.XX.X-X-default` – параметры сборки ядра
- `message` – интерфейс загрузчика

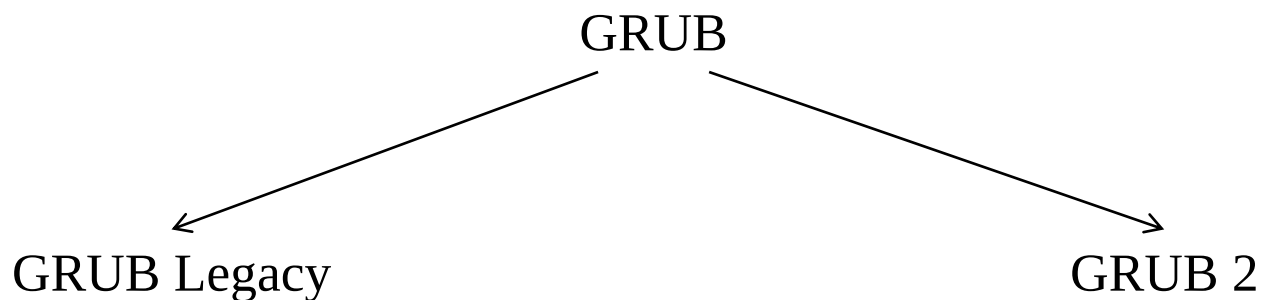
ЗАГРУЗЧИК LILO



ЗАГРУЗЧИК LILO

- При установке сохраняет старый MBR в файле */boot/boot.xxu*
- Местоположения ядра и вторичного загрузчика хранится в карте файлов */boot/map*
- Файл конфигурации */etc/lilo.conf*
- Ядро располагается в файле */boot/vmlinuz-up*
- После каждого изменения настроек требуется вызов утилиты */sbin/lilo*

ЗАГРУЗЧИК GRUB



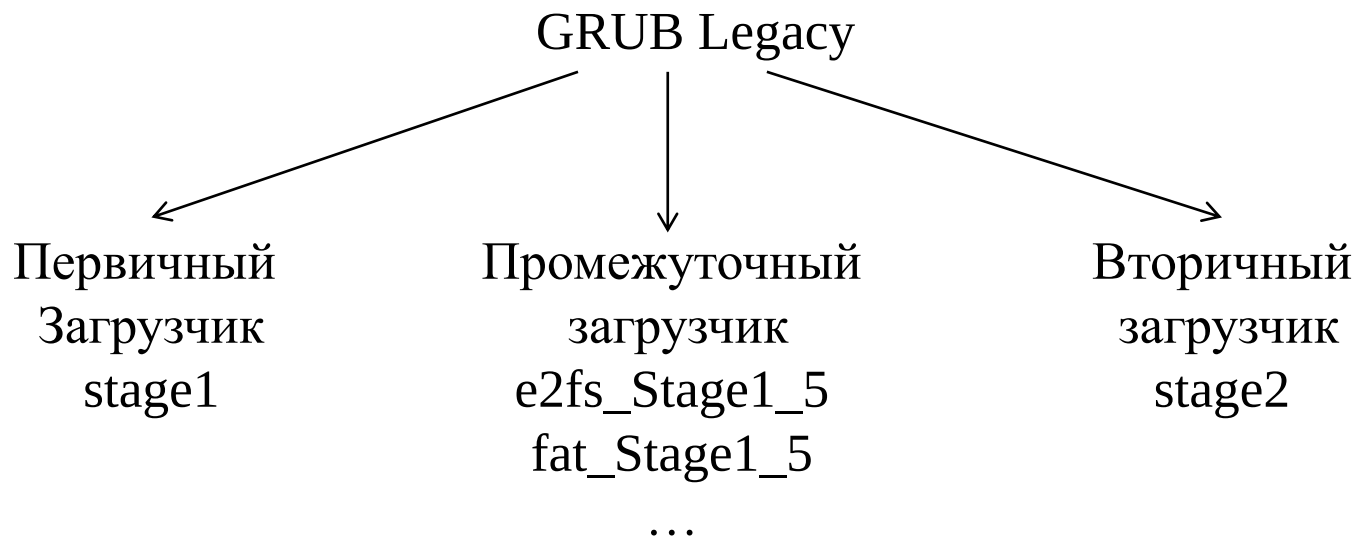
Первая версия загрузчика
(v1.x)

Основана на нескриптовой
загрузке

Вторая версия загрузчика
(v2.x)

Использует настраиваемые
скрипты

ЗАГРУЗЧИК GRUB Legacy



- размер 512 байт
- располагается в MBR или загрузочном секторе раздела
- загружает вторичный или полуторный загрузчики, используя карту размещения

- располагается за первичным загрузчиком
- представляет собой небольшой драйвер файловой системы
- загружает вторичный загрузчик по его имени в ФС

- располагается за первичным загрузчиком
- представляет собой небольшой драйвер файловой системы
- загружает вторичный

ЗАГРУЗЧИК GRUB Legacy

Особенности:

- Местоположения ядра и вторичного загрузчика располагаются в ФС и находятся по имени
- После каждого изменения настроек (!) не требуется запускать дополнительные утилиты

Файлы:

- */boot/backup_mbr* – резервная копия MBR
- */boot/grub/menu.lst* – файл настроек GRUB Legacy
- */boot/grub/xxx_Stage1_5* – полуторный загрузчик

НАСТРОЙКИ GRUB LEGACY



Основным файлом настроек является `/boot/grub/menu.lst`

- Записи группируются блоками
- Один блок соответствует одному варианту загрузки

Параметры блока:

- **title** – заголовок, отображающийся в меню выбора ОС
- **root** – идентификатор устройства, на котором расположено ядро ОС
- **kernel** – файловый путь до ядра ОС
- **sole** – время в секундах, в течение которого ожидается выбор пункта
- **initrd** – файловый путь до RAM-диска ОС (опционально)

НАСТРОЙКИ GRUB LEGACY

```
## specify if running in Xen domU or have grub detect automatically
## update-grub will ignore non-xen kernels when running in domU and vice versa
## e.g. indomU=detect
##     indomU=true
##     indomU=false
# indomU=true

## should update-grub create memtest86 boot option
## e.g. memtest86=true
##     memtest86=false
# memtest86=true

## should update-grub adjust the value of the default booted system
## can be true or false
# updatedefaultentry=false

## should update-grub add savedefault to the default options
## can be true or false
# savedefault=false

## ## End Default Options ##

title          Ubuntu 16.04.3 LTS, kernel 4.4.0-87-generic
root           (hd0)
kernel         /boot/vmlinuz-4.4.0-87-generic root=UUID=600e695e-e423-4733-9a30-d90ac9252d35 ro con
sole=hvc0
initrd         /boot/initrd.img-4.4.0-87-generic

title          Ubuntu 16.04.3 LTS, kernel 4.4.0-87-generic (recovery mode)
root           (hd0)
kernel         /boot/vmlinuz-4.4.0-87-generic root=UUID=600e695e-e423-4733-9a30-d90ac9252d35 ro si
ngle
initrd         /boot/initrd.img-4.4.0-87-generic

### END DEBIAN AUTOMAGIC KERNELS LIST
root@testserver:~#
```

ЗАГРУЗЧИК GRUB2

Особенности:

- Местоположения ядра и вторичного загрузчика располагаются в ФС и находятся по имени
- После каждого изменения настроек необходимо пересобрать с помощью `grub2-mkconfig`

Файлы:

- `/boot/backup_mbr` – резервная копия MBR
- `/etc/default/grub` – файл настроек GRUB2
- `/boot/grub2/grub.cfg` – конфигурационный скрипт
- `/etc/grub.d` – директория со скриптами, участвующими в формировании `grub.cfg`

КОНФИГУРАЦИОННЫЕ СКРИПТЫ GRUB2

Скрипты в `/etc/grub.d`:

- `00_header` – обрабатывает `/etc/default/grub`
- `10_linux` – осуществляет поиск ядра
- `20_linux_xen` – поиск хен-ядра
- `30_os_prober` – проверка наличия необходимых частей ОС
- `40_custom` – скрипт для добавления пользовательских элементов



В `/etc/grub.d` можно добавлять собственные скрипты, расширяя загрузчик



XX_ в начале каждого имени скрипта это порядковый номер обработки и исполнения при загрузке

НАСТРОЙКИ GRUB2



Основным файлом настроек является `/etc/default/grub`

Параметры :

- **GRUB_DEFAULT** – определяет какой пункт загружается по умолчанию
- **GRUB_HIDDEN_TIMEOUT** – сколько секунд отображается пустой экран
- **GRUB_TIMEOUT** – время в секундах, в течение которого ожидается выбор пункта меню
- **GRUB_DISTRIBUTOR** – название дистрибутива
- **GRUB_CMDLINE_LINUX_DEFAULT** – параметры ядра при обычной загрузке
- **GRUB_TERMINAL** – включает консольный режим и отключает графику
- **GRUB_GFXMODE** – задает экранное разрешение меню
- **GRUB_DISABLE_RECOVERY** – отключает меню GRUB2

НАСТРОЙКИ GRUB2

```
# If you change this file, run 'grub2-mkconfig -o /boot/grub2/grub.cfg' afterwards to update
# /boot/grub2/grub.cfg.

# Uncomment to set your own custom distributor. If you leave it unset or empty, the default
# policy is to determine the value from /etc/os-release
GRUB_DISTRIBUTOR=
GRUB_DEFAULT=saved
GRUB_HIDDEN_TIMEOUT=0
GRUB_HIDDEN_TIMEOUT_QUIET=true
GRUB_TIMEOUT=8
GRUB_CMDLINE_LINUX_DEFAULT="resume=/dev/sda1 splash=silent quiet showopts"
GRUB_CMDLINE_LINUX=""

# Uncomment to automatically save last booted menu entry in GRUB2 environment

# variable `saved_entry'
# GRUB_SAVEDefault="true"
#Uncomment to enable BadRAM filtering, modify to suit your needs

# This works with Linux (no patch required) and with any kernel that obtains
# the memory map information from GRUB (GNU Mach, kernel of FreeBSD ...)
# GRUB_BADRAM="0x01234567,0xfefefefe,0x89abcdef,0xefefefef"
#Uncomment to disable graphical terminal (grub-pc only)

GRUB_TERMINAL="gfxterm"
# The resolution used on graphical terminal
#note that you can use only modes which your graphic card supports via VBE

# you can see them in real GRUB with the command `vbeinfo'
GRUB_GFXMODE="auto"
# Uncomment if you don't want GRUB to pass "root=UUID=xxx" parameter to Linux
# GRUB_DISABLE_LINUX_UUID=true
#Uncomment to disable generation of recovery mode menu entries

# GRUB_DISABLE_LINUX_RECOVERY="true"
#Uncomment to get a beep at grub start
```

КОМАНДА GRUB-INSTALL

Описание: установка или восстановление загрузчика GRUB2

Формат:

```
grub-install [KEYS] DEVICE
```

Ключи:

--boot-directory DIR — задать в качестве загрузочной директорию DIR

--force — игнорировать ошибки выполнения

--no-bootsector — не изменять MBR

--recheck — принудительно перезаписывать /boot/grub2/device.map

--removable — установка на съемный носитель

--version — определить версию загрузчика

КОМАНДА GRUB2-MKCONFIG

Описание: генерация конфигурационного файла для GRUB2

Формат:

```
grub2-mkconfig [KEYS]
```

Ключи:

-o FILE (**--outout=FILE**) — задать положение выходного файла

--version — вывести версию GRUB2 и самой утилиты

Пример:

```
# генерация нового /boot/grub2/grub.cfg
grub2-mkconfig -o /boot/grub2/grub.cfg
```

ЯДРО LINUX



- Ядро располагается в файле `vmlinuz-X.XX.X-X`
- Ядро представляет собой архив, созданный при помощи программы `gunzip`

ЯДРО LINUX

Особенности ядра Linux:

- Поддерживает динамическую загрузку модулей ядра
- Все подзадачи ядра реализованы в виде потоков, которые могут иметь общие ресурсы
- Потоки могут быть вытеснены более приоритетной задачей, даже в режиме ядра
- Поддерживает симметричную многопроцессорную обработку (SMP)
- Легко масштабируемо

RAM-ДИСК

Initrd (**IN**itial **R**am **D**isk) – образ виртуальной файловой системы, создаваемый при установке ОС (или через команду `mkinitrd`)

- Initrd используется для подключения к ядру дополнительных модулей
- Располагается в файле `/boot/initrd`
- Initrd представляет собой архив, созданный при помощи программы *gunzip* и *cpio*

РАМ-ДИСК

Просмотр содержимого initrd:

```
mkdir    temp
cd temp

# скопировать  initrd-2.6.18.2-34-default в /temp/
# initrd.img.gz
cp /boot/initrd-2.6.18.2-34-default initrd.img.gz

# распаковать  initrd-.6.18.2.img.gz
gunzip initrd-2.6.18.2.img.gz

# распаковать  initrd-.6.18.2.img
cpio -i --make-directories < initrd-2.6.14.2.img
```

КОМАНДА MKINITRD

Описание: создание initrd из элементов запущенной системы

Формат:

```
mkinitrd [KEYS]
```

Ключи:

- b** DIR – задать директорию, в которой initrd будет создан
- d** DEV – задать устройство, на которое смонтирована корневая ФС
- A** – создать initrd, содержащий все файлы и модули текущей ОС
- version** – определить версию загрузчика

Пример:

```
# генерация нового initrd в ~/myboot  
mkdir ~/myboot && mkinitrd -b ~/myboot -d /dev/sda2
```


LINUX-СТАРТЕРЫ

Системы инициализации

System V

- На основе bash-скриптов
- Поддерживает уровни доступа
- Уровни доступа – директории со скриптами

BSD

- На основе bash-скриптов
- Не поддерживает уровни доступа

Systemd

- Но на основе конфигурационных файлов
- Unit-ориентированная
- Поддерживает уровни доступа на основе целей

ЗАГРУЗКА ЧЕРЕЗ SYSTEM V

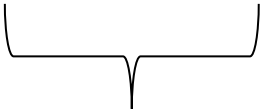
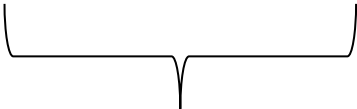
ЗАПУСК КОРНЕВОГО ПРОЦЕССА

- Запуск корневого процесса */sbin/init* и выполнение инструкций */etc/inittab*
- Создание псевдо-файловых систем (*/proc*) : **mount -n -t proc /proc /proc**
- Проверка корневой ФС на наличие ошибок : **fsck -T -A -a**
- Перемонтирование корневой ФС в режиме «чтение-запись» : **mount -n -o remount,rw /**
- Монтирование всех ФС, перечисленных в */etc/fstab* : **mount -a**
- Монтирование swap-раздела : **swapon /dev/sda1**
- Переход в режим заданного уровня загрузки (runlevel) и запуск стартовых скриптов :
- Запуск виртуального терминала

КОРНЕВОЙ ПРОЦЕСС

- После запуска *init* просматривает свой конфигурационный файл */etc/inittab*

Формат */etc/inittab* :

<Label>	:	<Runlevel>	:	<Mode>	:	<Process>
						
Метка записи		Уровень загрузки		Режим запуска процесса		Исполняемый файл процесса

КОРНЕВОЙ ПРОЦЕСС

```
# The default runlevel is defined here
id:5:initdefault:

# First script to be executed, if not booting in emergency (-b) mode
si::bootwait:/etc/init.d/boot

# /etc/init.d/rc takes care of runlevel handling
#
# runlevel 0 is System halt (Do not use this for initdefault!)
# runlevel 1 is Single user mode
# runlevel 2 is Local multiuser without remote network (e.g. NFS)
# runlevel 3 is Full multiuser with network
# runlevel 4 is Not used
# runlevel 5 is Full multiuser with network and xdm
# runlevel 6 is System reboot (Do not use this for initdefault!)
#
10:0:wait:/etc/init.d/rc 0
11:1:wait:/etc/init.d/rc 1
12:2:wait:/etc/init.d/rc 2
13:3:wait:/etc/init.d/rc 3
#14:4:wait:/etc/init.d/rc 4
15:5:wait:/etc/init.d/rc 5
16:6:wait:/etc/init.d/rc 6

# what to do in single-user mode
ls:S:wait:/etc/init.d/rc S
~~:S:respawn:/sbin/sulogin

# what to do when CTRL-ALT-DEL is pressed
ca::ctrlaltdel:/sbin/shutdown -r -t 4 now

# special keyboard request (Alt-UpArrow)
# look into the kbd-0.90 docs for this
kb::kbrequest:/bin/echo "Keyboard Request -- edit /etc/inittab to let this work."

# what to do when power fails/returns
pf::powerwait:/etc/init.d/powerfail start
pn::powerfailnow:/etc/init.d/powerfail now
#pn::powerfail:/etc/init.d/powerfail now
po::powerokwait:/etc/init.d/powerfail stop
```

УРОВНИ ЗАГРУЗКИ

Runlevel 0: остановка системы

Runlevel 1: однопользовательский режим; не запускаются службы, работает одна консоль

Runlevel 2: многопользовательский режим; не запускаются сетевые службы

Runlevel 3: многопользовательский сетевой режим; запущены сетевые службы и сервера

Runlevel 4: не используется

Runlevel 5: многопользовательский графический режим; запущены графические службы

Runlevel 6: перезагрузка системы

ИЗМЕНЕНИЕ УРОВНЯ ЗАГРУЗКИ

Способ 1: вызов init

```
# переход в 3-й runlevel  
init 3
```

Способ 2: редактирование /etc/inittab

```
# переход в 3-й runlevel  
id:3:initdefault:
```

Способ 3: передача параметра ядру

```
# добавляем в 40_custom параметр ядра  
module vmlinuz-X.XX.X-X root=/dev/sda2 ro 3 quit
```

СТАРТОВЫЕ СКРИПТЫ

init.d – директория, содержащая startup-скрипты для уровней доступ

Содержимое **/etc/init.d**:

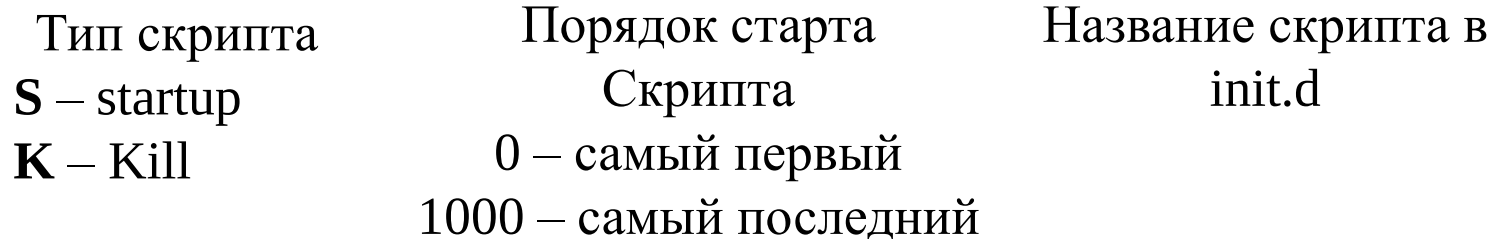
- *rcX.d* –директория для скриптов *runleveX*
- *rc4.d* – пустая директория для формирвоания *runlevel4*
- *rcS.d* – скрипты, запускаемые по умолчанию всегда
- Startup-скрипты



rc.d – символическая ссылка на init.d оставленная для совместимости

СТАРТОВЫЕ СКРИПТЫ

S10-mailman



Скрипты из rcX.d – это символичные ссылки на скрипты из init.d, которым при старте передается соответствующий параметр –S или –K

ЗАГРУЗКА ЧЕРЕЗ SYSTEMD

ТИПЫ ЮНИТОВ (UNIT)

- **/etc/systemd/system/** — юниты, созданные системным администратором для формирования уровней доступа
- **/run/systemd/system/** — юниты, которые созданы в процессе исполнения приложений или после установки
- **/usr/lib/systemd/system/** — юниты, созданные при установке приложений и сервисов



Поиск юнитов осуществляется в директориях в порядке **/etc...**, **/run...**, **/usr...**



Юниты, размещенные в директориях **/usr/lib/systemd/system/** и **/run/systemd/system/** не нужно редактировать

ТИПЫ УДИНИЦ (UNIT)

- **service** — запускает, останавливает или перезагружает демоны, также можно запускать SysV-сценарии
- **socket** — конфигурационный файл сокета, который связан с определенным сервисом
- **device** — содержит правило для обработки дерева устройств
- **mount** — монтирования файловой системы
- **automount** — автоматическое монтирование файловой системы
- **target** — логическая группировка единиц, ссылается на другие юниты
- snapshot** — создание ссылок на другие юниты
- **timer** — активация юнитов по таймеру
- **swap** — управление файлами подкачки
- **path** — активация других служб

СТРУКТУРА UNIT-ФАЙЛА

[Section_Name]

Variable1=Value1

.

VariableN=ValueN

Пример:

```
# описание главной секции unit-файла
```

[Unit]

Description=MyUnit

After=syslog.target

After=network.target

After=nginx.service

СЕКЦИЯ UNIT

- **Description** — описание юнита:

Description=MyUnit

- **After** — запуск юнита после сервиса или группы сервисов:

After=cron.service; After=user.target

- **Requires** — необходимый для запуска юнит:

Requires=cron.services

- **Wants** — желаемый для запуска юнит:

Wants=http.service

СЕКЦИЯ SERVICE

- **Type** — тип юнита:
 - service – служба запускается незамедлительно
 - fork – служба соблюдает очередность и создается расщеплением родителя
- **WorkingDirectory** — рабочая директория сервиса
- **User** — от какого пользователя следует запускать
- **Group** — от какой группы следует запускать
- **ExecStart** — команда на старт сервиса
- **ExecStop** — команда на остановку сервиса
- **ExecReload** — команда на перезапуск сервиса
- **TimeOut** — сколько секунд система ожидает старта/перезапуска сервиса

СЕКЦИЯ INSTALL

- **WantedBy** — определение группы юнитов, к которой относится текущий

КОМАНДА SYSTEMCTL

Описание: управление объектами ФС и службами

Формат:

```
systemctl [COMMAND] [KEYS] [SERVICE | TARGET]
```

Ключи:

--type=TYPE – фильтрация юнитов типа TYPE

--state=STATE – фильтрация юнитов по статусу STATE

--property=PROP – фильтрация юнитов по свойству PROP

--all – показать все юниты со статусом LOADED

--failed – список служб, которые не загрузились

КОМАНДА SYSTEMCTL

Команды:

list-unit-files — список юнитов, установленных в системе

start — однократно запустить юнит

stop — однократно остановить юнит

enable — сделать юнит загружаемым автоматически при старте ОС

disable — убрать юнит из автозагрузки

is-enabled — проверить является ли юнит загружаемым при старте ОС

КОМАНДА SYSTEMCTL

Пример:

```
# вывести список запущенных юнитов
```

```
Systemctl
```

```
# вывести список целей
```

```
Systemctl list-units --type=target
```

```
# вывести список юнитов, завершившихся аварийно
```

```
Systemctl --failed
```

```
# вывести список загруженных юнитов
```

```
systemctl --state=loaded
```

```
# сделать цель graphic.target загружаемой автоматически
```

```
systemctl enable graphic.target
```

```
# запустить юнит sshd.service
```

```
systemctl start sshd.service
```

УРОВНИ ЗАГРУЗКИ

Уровень	Цель systemd	Описание
0	<code>runlevel0.target,</code> <code>poweroff.target</code>	Остановка системы
1, s, single	<code>runlevel1.target,</code> <code>rescue.target</code>	Однопользовательский режим (single user mode)
2	<code>runlevel2.target,</code> <code>multi-user.target</code>	Пользовательский уровень. Как и 3-й но без поддержки сети
3	<code>runlevel3.target,</code> <code>multi-user.target</code>	Многопользовательский режим (multiuser mode) с поддержкой сети
4	<code>runlevel4.target,</code> <code>multi-user.target</code>	Не используется
5	<code>runlevel5.target,</code> <code>graphical.target</code>	Многопользовательский графический режим с поддержкой сети
6	<code>runlevel6.target,</code> <code>reboot.target</code>	Перезагрузка системы
emergency	<code>emergency.target</code>	Аварийный режим, аналогом 1-го уровня

ИЗМЕНЕНИЕ УРОВНЯ ЗАГРУЗКИ

Способ 1: вызов systemctl

```
# переход в 3-й runlevel  
systemctl enable multi-user.target
```

Способ 2: редактирование default.target

```
# переход в 3-й runlevel  
ln -sf /usr/lib/systemd/system/multi-user.target  
/etc/systemd/system/default.target
```

Способ 3: передача параметра ядру

```
# добавляем в 40_custom параметр ядра  
module vmlinuz-X.XX.X-X root=/dev/sda2 ro 3 quit
```